Bayesian Inference & Phylogenetics

BOT6935/ZOO6927 Phylogenomics

Spring 2018

1. Software we'll cover in lab today:

1. MrBayes

2. Getting onto the server

1. Log on to Hipergator (`ssh <username>@hpg2.rc.ufl.edu`)
2. Move to the directory with your class files using the appropriate command:

   ```
   $ cd /ufrc/<group>/<user>
   $ cd /ufrc/zoo6927/share/<user>
   ```

3. Make a new directory for today's lab and move into it:

   ```
   $ mkdir MrBayes
   $ cd MrBayes
   ```

4. Copy (using the `cp` command) the following files into your MrBayes directory. Ask Emily if you need help with the command.

- /ufrc/zoo6927/share/mrbayes/primates.nex
- /ufrc/zoo6927/share/mrbayes/primatesMrB.nex
- /ufrc/zoo6927/share/mrbayes/mrbayes.mpi.slurm

1. Bayesian inference with MrBayes - Interactive

Much of this section comes from a MrBayes lab written by David Baum at the University of Wisconsin-Madison, for BOT 563

We will first use MrBayes interactively, and then through a submission script, to do some Bayesian phylgenetic analyses.

To run MrBayes interactively, we need to request time from the server. You MUST do this before running a program like MrBayes interactively from the command line, rather than through a submission script, so that the server can manage your request appropriately.

```
module load ufrc
srundev --time=01:00:00
```

This loades the module `ufrc` and requests 1 hour of interactive time. Once this has successfully executed, you will need to `cd` back to your directory with the MrBayes files in it:

```
cd /ufrc/zoo6927/share/<user>/MrBayes
```

Next you need to load the modules that are necessary to run MrBayes:

```
module load intel/2016.0.109
module load openmpi/1.10.2
module load mrbayes/3.2.6
```

And now we're ready to actually execute the file and work with Mrbayes. First, open the file in TextWrangler and take a look at it. This is a typical nexus file, which is one of the standard file types in phylogenetics. At the top is some information about the dimensions of the dataset, including information about the number of taxa and characters, followed by the actual data matrix itself. Now go ahead and execute the file in MrBayes on the command line:

```
mpiexec mb -i primates.nex
```

The `-i` flag tells it to run in interactive mode. After executing this command, you should see some evidence that MrBayes has successfully read in your file and found your taxa and character blocks (look at the nexus file to confirm what's in there). If it does not open or run, chances are that you are in the wrong directory. Use `pwd` to check where you are, and `ls -l` to see a list of files in the directory where you are. Confirm that you are in the right place, and that the primates.nex file is also in that directory, BEFORE you ask for help if you are having trouble at this step!

Once MrBayes has successfully read in your file, next you want to start a log file:

```
MrBayes > log start filename=primates.log append
```

Setting up the priors, models, and analysis

Next we need to set the model that we want to use. We will use GTR+I+G as the nucleotide substitution model, though you can configure any model in MrBayes using the correct commands. A list of some of them can be found at this website (https://gist.github.com/brantfaircloth/895282); I suggest you keep it handy if you are going to be running Bayesian phylogenetic analyses! The command `lset` will specify the model:

```
MrBayes > lset nst=6 rates=invgamma
```

There are six types of parameters in the model: the topology, the branch lengths, the four stationary frequencies of the nucleotides, the six different nucleotide substitution rates, the proportion of invariable sites, and the shape parameter of the gamma distribution of rate variation. The default priors in MrBayes work well for most analyses, and we will not change any of them here. Let's check the model before we proceed:

```
MrBayes > showmodel
```

Next we want to set up the mcmc analysis. The `mcmcp` command does this, and sets the number of generations the analysis should run for, and tells it to print to screen every 100 generations:

```
MrBayes > mcmcp ngen=10000 printfreq=100 samplefreq=10
```

Now run the analysis:

```
MrBayes > mcmc
```

MrBayes will first print information about the model and then list the proposal mechanisms that will be used in sampling from the posterior distribution. Here is one example, but these values will change. Basically this tells you the types of changes it will propose to the various parameters, and with what frequency. (If you want to know more about the moves and what each one specifies, once your analysis is finished enter `showmoves` and it will list them all with explanations.)

```
The MCMC sampler will use the following moves:
      With prob.  Chain will use move
         1.09 %   Dirichlet(Revmat)
         1.09 %   Slider(Revmat)
         1.09 %   Dirichlet(Pi)
         1.09 %   Slider(Pi)
         2.17 %   Multiplier(Alpha)
         2.17 %   Slider(Pinvar)
        10.87 %   ExtSPR(Tau,V)
        10.87 %   ExtTBR(Tau,V)
        10.87 %   NNI(Tau,V)
```

```
10.87 %   ParsSPR(Tau,V)
32.61 %   Multiplier(V)
10.87 %   Nodeslider(V)
 4.35 %   TLMultiplier(V)
```

MrBayes will spend most of its effort changing topology and branch lengths since these are the most difficult parameters to integrate over.

After the initial log likelihoods, MrBayes will print to screen the state of the chains every 100th generation; it should look something like this:

```
   Chain results:
       1 -- (-7812.831) (-7523.685) [-7485.569] (-7700.309) * [-7832.045] (-7618.595) (-7776.608) (-7836.826)
     100 -- (-6771.532) (-6857.529) (-6766.678) [-6682.527] * [-6506.277] (-6944.449) (-6784.126) (-6991.307) -- 0:01:39
     200 -- (-6321.464) [-6179.561] (-6338.168) (-6272.242) * (-6339.400) (-6715.840) [-6265.329] (-6599.698) -- 0:01:38
     300 -- (-6201.285) (-6084.899) (-6139.200) [-6049.061] * [-6073.056] (-6359.134) (-6106.834) (-6515.348) -- 0:01:37

…

    1000 -- (-5811.949) [-5737.884] (-5888.234) (-5819.793) * (-5867.377) (-5851.693) (-5784.437) [-5749.264] -- 0:01:21
   Average standard deviation of split frequencies: 0.073946
    1100 -- (-5784.586) [-5730.069] (-5880.476) (-5798.728) * (-5839.268) (-5836.074) (-5779.940) [-5739.170] -- 0:01:20
...
...
    9900 -- (-5743.556) [-5724.246] (-5736.484) (-5731.803) * [-5727.082] (-5732.476) (-5728.482) (-5725.045) -- 0:00:00
   10000 -- (-5738.106) [-5723.380] (-5726.322) (-5727.599) * (-5728.776) (-5732.410) [-5728.515] (-5725.418) -- 0:00:00
Average standard deviation of split frequencies: 0.0001
Continue with analysis? (yes/no):
```

The first column lists the generation number. The following four columns with negative numbers each correspond to one chain in the first run. The default setting is to do two runs simultaneously, each with 1 cold chain and 3 hot chains. Each column corresponds to one physical location in computer memory, and the chains shift positions in the columns as the run proceeds. The numbers are the log likelihood values of the chains. The chain that is currently the cold chain has its value surrounded by square brackets [ ], whereas the heated chains have their values surrounded by parentheses (). When two chains successfully change states, they trade column positions (places in computer memory). If the Metropolis coupling works well, the cold chain should move around among the columns; this means that the cold chain successfully swaps states with the heated chains. If the cold chain gets stuck in one of the columns, then the heated chains are not successfully contributing states to the cold chain, and the Metropolis coupling is inefficient. The analysis may then have to be run longer or the temperature difference between chains may have to be lowered.

The star column separates the two different runs. The last column gives the time left to completion of the specified number of generations. Different moves are used in each generation, so the exact time varies somewhat for each set of 100 generations, and the predicted time to completion will be unstable in the beginning of the run. After a while, the predictions will become more accurate and the time will decrease predictably. Once it has run the 10,000 generations, it will ask if you wish to proceed…

When to stop the analysis

Although it is advisable to use convergence diagnostic, such as the standard deviation of split frequencies, to determine run length, there are simpler but less powerful methods of determining when to stop the analysis. Arguably the simplest technique is to examine the log likelihood values (or, more exactly, the log probability of the data given the parameter values) of the cold chain, that is, the values printed to screen within square brackets. In the beginning of the run, the values typically increase rapidly (the absolute values decrease, since these are negative numbers). This phase of the run is referred to as the "burn-in" and the samples from this phase are typically discarded. Once the likelihood of the cold chain stops to increase and starts to randomly fluctuate within a more or less stable range, the run may have reached stationarity, that is, it is producing a good sample from the posterior probability distribution. At stationarity, we also expect different, independent runs to sample similar likelihood values. Trends in likelihood values can be deceiving though; you're more likely to detect problems with convergence by comparing split frequencies than by looking at likelihood trends.

When it asks whether or not you want to continue with the analysis, examine the average standard deviation of split frequencies. As the two runs converge onto the stationary distribution, we expect the average standard deviation of split frequencies to approach zero, reflecting the fact that the two tree samples become increasingly similar. In our case, the average standard deviation is about 0.07 after 1,000 generations and then drops drastically towards the end of the run. Values can differ slightly because of stochastic effects. Given the low value of the average standard deviation at the end of the run (and that this is a sample exersise), there appears to be no need to continue the analysis beyond 10,000 generations.

To stop the analysis, type `no` when MrBayes asks "Continue with analysis? (yes/no):

When you stop the analysis, MrBayes will print several types of information useful in optimizing the analysis. This is primarily of interest if you have difficulties in obtaining convergence. Since we apparently have a good sample from the posterior distribution already (based on standard deviation of split frequencies), we will ignore this information for now.

Summarizing Samples of Substitution Model Parameters

During the run, samples of the substitution model parameters have been written to the .p files every `samplefreq` generation. These files are tab-delimited text files that look something like this:

```
[ID: 5848203808]
  Gen  LnL        TL     r(A<->C)  ...  pi(G)     pi(T)     alpha     pinvar
    1  -7559.137  2.044  0.166667  ...  0.250000  0.250000  0.500000  0.000000
   10  -6585.519  2.181  0.090180  ...  0.169513  0.247418  0.569271  0.036162
   ...
 9990  -5728.935  2.527  0.051106  ...  0.076213  0.262107  0.932771  0.194004
10000  -5722.857  2.642  0.051106  ...  0.078383  0.246791  0.721217  0.185486
```

The first number, in square brackets, is a randomly generated ID number that lets you identify the analysis from which the samples come. The next line contains the column headers, and is followed by the sampled values. From left to right, the columns contain: 1. The generation number (Gen) 2. The log likelihood of the cold chain (LnL) 3. The total tree length (the sum of all branch lengths, TL) 4. The six GTR rate parameters (r(A<->C), r(A<->G) etc) 5. The four stationary nucleotide frequencies (pi(A), pi(C) etc) 6. The shape parameter of the gamma distribution of rate variation (alpha) 7. The proportion of invariable sites (pinvar). If you use a different model for your data set, the .p files will of course be different.

1. How many samples (lines) should there be in the .p file for this analysis?

Next, type:

```
MrBayes > sump relburnin=no
```

The sump command will first generate a plot of the generation versus the log probability of the data (the log likelihood values). We want to see the entire analysis, from generation 0 through 10,000, so we tell it not to automatically remove any burnin (this is `relburnin=no`). If we are at stationarity, this plot should eventually (towards the right-hand side) look like 'white noise', that is, there should be no tendency of increase or decrease over time.

2. Sketch the plot below or on a blank sheet of paper.

The burnin is a fixed number of samples from the beginning of the chain to be discarded. Since we sampled every 10th generation, there are 1,000 samples (1,001 to be exact, since the first generation is always sampled).

3. Based on your plot, how many generations did it take to reach stationarity? _____

4. So, what would be an appropriate "burnin" (in number of samples)? _____

Type the following, replacing `xxx` with your number from 4.

```
MrBayes > sump burnin=xxx
```

Look at the resulting plot and decide whether that really was enough burnin. If not, adjust your answer above. If you see any obvious trend in your plot, either increasing or decreasing, you probably need to run the analysis longer to get an adequate sample from the posterior probability distribution.

At the bottom of the sump output, there is a table summarizing the samples of the parameter values:

```
Model parameter summaries over the runs sampled in files
   "primates.nex.run1.p" and "primates.nex.run2.p":
   (Summaries are based on a total of 1502 samples from 2 runs)
   (Each run produced 1001 samples of which 751 samples were included)
                                          95% Cred. Interval
                                       ----------------------
Parameter       Mean        Variance       Lower         Upper         Median       PSRF *
-------------------------------------------------------------------------------------------
TL           2.885314      0.071446      2.408000      3.464000      2.861000      1.004
r(A<->C)     0.045110      0.000070      0.031406      0.061424      0.044756      0.999
r(A<->G)     0.477554      0.002421      0.387141      0.569168      0.473521      1.040
r(A<->T)     0.038541      0.000060      0.024204      0.053787      0.038402      0.999
r(C<->G)     0.033765      0.000195      0.010836      0.064049      0.032475      1.033
r(C<->T)     0.386144      0.001735      0.311504      0.459011      0.386897      1.012
r(G<->T)     0.018885      0.000133      0.001217      0.045924      0.016801      1.048
pi(A)        0.355301      0.000162      0.329769      0.377153      0.356808      1.092
pi(C)        0.320831      0.000131      0.299953      0.341757      0.321312      1.152
pi(G)        0.081154      0.000048      0.068816      0.098442      0.081723      1.014
pi(T)        0.242714      0.000104      0.226573      0.264684      0.242113      1.000
alpha        0.714405      0.037305      0.419210      1.147286      0.673545      1.007
pinvar       0.185901      0.004887      0.025889      0.307468      0.188027      1.009
-------------------------------------------------------------------------------------------
   * Convergence diagnostic (PSRF = Potential scale reduction factor [Gelman and Rubin, 1992],
uncorrected) should approach 1 as runs converge. The values may be unreliable if you have a small
number of samples. PSRF should only be used as a rough guide to convergence since all the
assumptions that allow one to interpret it as a scale reduction factor are not met in the
phylogenetic context.
```

For each parameter, the table lists the mean and variance of the sampled values, the lower and upper boundaries of the 95% credibility interval, and the median of the sampled values. The parameters are the same as those listed in the .p files: the total tree length (TL), the six reversible substitution rates (r(A<- >C), r(A<->G), etc), the four stationary state frequencies (pi(A), pi(C), etc), the shape of the gamma distribution of rate variation across sites (alpha), and the proportion of invariable sites (pinvar). Note that the six rate parameters of the GTR model are given as proportions of the rate sum (the Dirichlet parameterization).

The last column in the table contains a convergence diagnostic, the Potential Scale Reduction Factor (PSRF). If we have a good sample from the posterior probability distribution, these values should be close to 1.0. If you have a small number of samples, there may be some spread in these values, indicating that you may need to sample the analysis more often or run it longer. In our case, we can probably easily obtain more accurate estimates of some parameters by running the analysis slightly longer.

Summarizing Tree and Branch Length Information

```
MrBayes > sumt burnin=250
```

The sumt command will output, among other things, summary statistics for the taxon bipartitions, a tree with clade credibility (posterior probability) values, and a phylogram (if branch lengths have been saved). The summary statistics (see below) describes each partition in the "dot-star" format (dots for the taxa that are on one side of the partition and stars for the taxa on the other side). Then it gives the number of times the partition was sampled (#obs), the probability of the partition (Probab.), the standard deviation of the partition frequency (Stdev(s)), the mean (Mean(v)) and variance (Var(v)) of the branch length, the Potential Scale Reduction Factor (PSRF), and the number of runs in which the partition was sampled (Nruns). In this analysis, there is overwhelming support for one tree, so all partitions have a posterior probability of 1.0 or very close to it.

The clade credibility tree gives the probability of each partition or clade in the posterior distribution, and the phylogram gives the branch lengths measured in expected substitutions per site, averaged across the posterior distribution.

In the background, the sumt command creates three additional files that you can see in Cyberduck (or PC equivalent). The first is a .parts file, which contains the list of taxon bipartitions, their posterior probabilities (the proportion of sampled trees containing them), and the branch lengths associated with them (if branch lengths have been saved). The branch length values are based only on those trees containing the relevant bipartition. The second generated file has the suffix .con and includes two consensus trees. The first one has both the posterior probability of clades (as interior node labels) and the branch lengths (if they have been saved) in its description. The second tree only contains the branch lengths. The third file generated by the sumt command is the .trprobs file, which contains the trees that were found during the MCMC search, sorted by posterior probability.

To quit out of the interactive version of MrBayes, type `quit` at its internal prompt, and you will be returned to the standard terminal prompt (`$`).

6. Bayesian inference with MrBayes - Submitting a job to the HPC

MrBayes/nexus pro-tip: note that in the nexus file, every line MUST end with a ;! If your file is not executing, or you are having issues with it, check to make sure that in the MrBayes block you remembered to close every line with a semi-colon. I promise this will save you grief.

For this part you'll use the `primatesMrB.nex` and `mrbayes.mpi.slurm` files. First, open the slurm submission script, change the email address, and then compare the commands with the ones you entered above to start MrBayes. They are identical, except that we are no longer telling it to run interactively.

Next, open up the `primatesMrB.nex` nexus file. This file is identical to the previous `primates.nex` file, except that now there is a MrBayes block at the bottom. You can use the exact format of this block in your own analyses, changing the parts that are relevant for you. This block is where you put all the commands that you would have entered interactively; it reads them all in and does everything you ask one line at a time as it processes your file.

Let's submit the job and then look at the nexus file while it's running:

```
sbatch mrbayes.mpi.slurm
```

Here is some explanation of the different sections:

```
 charset cox1 = 1 - 300;
 charset cox2 = 301 - 550;
 charset cox3 = 551 - 898;

partition genes = 3: cox1, cox2, cox3;
set partition = genes;
```

This first section defines the partitions within the dataset and gives them names, then tells MrBayes to use this set of partitions. It's a little redundant, but you do need all of this text.

```
unlink statefreq=(all) revmat=(all) shape=(all) pinvar=(all);
prset applyto=(all) ratepr=variable;
```

The first line here tells MrBayes that the values for all of these parameters will be unlinked between partitions. That means, for example, that the proportion of invariable sites (`pinvar`) in one partition will be estimated separately from that in any other partition. The second line tells MrBayes to expect site specific rates that will be modeled, and modeled differently between partitions. The next two lines then provide the specific information about the models and which partitions they apply to...

```
lset applyto=(1,2) nst=6 rates=invgamma;  [GTR+I+G]
lset applyto=(3) nst=2 rates=gamma; [HKY+G]
```

The `lset applyto=()` tells which partition each model should apply to, and the remaining parts of each line define the specific model. Note that anything within brackets [ ] is a comment and is invisible to MrBayes; this is just for you to see which model is which. Here is that website again that lists how to code most of the GTR family of models in MrBayes: https://gist.github.com/brantfaircloth/895282

5. How would you write the lines so that partitions 1 and 3 have F81+G, and 2 has SYM+I?

```
constraint macaques -1 = Macaca_fuscata M_mulatta M_fascicularis M_sylvanus;
prset topologypr = constraints(macaques);
```

These lines set a constraint, meaning that the four macaques will be treated as a clade, and only trees in which they do form a clade will be considered in the analysis. The first line defines the constraint, and the second line tells the analysis to use it. You could set up multiple constraints by writing that line like this:
`prset topologypr = constraints(macaques, constraint2);`

```
mcmc ngen=50000 nruns=2 nchains=4 temp=0.200 swapfreq=1 nswaps=1 samplefreq=1000 savebrlens=yes
  stoprule=no;
sumt burnin=20;
```

And these last lines define the actual mcmc analysis, and the sumt command will save the files that include the final consensus tree.

When the analysis is finished, the output that printed to screen when you interacted with MrBayes will end up in the default .log file produced by HPG. The sumt output is there, and the consensus tree is the file that ends with .con.tre.

6. How do this tree and results compare to the interactive MrBayes run?

Assessing Chain Swap Frequency

In the log file produced by the server for your MrBayes run (file name format is: ..out), find the section with the chain swap information tables for the two runs (they are two small tables a little more than halfway down).

One way to verify that there is good mixing is to look at the table of acceptance rates of the proposal mechanisms used in your analysis. The Metropolis proposals used by MrBayes work best when their acceptance rate is neither too low nor too high. A rough guide is to try to get them within the range of 20% to 70%; rates outside this range are not necessarily a big problem but they typically mean that the analysis is inefficient.

Record the values from the chain swap information tables here, for both chains (values in the upper triangle):

1 -> 2: _____

2 -> 3: _____

3 -> 4: _____

If the mixing values are very low or very high, you can try using a much lower or a much higher temp (e.g., values around 0.01 and 0.8) and then reassess to see the effects on the results. For example, if the values are low, indicating not enough swapping, you might try a lower temperature so that the landscape of treespace will have more topography, and the chain might be more likely to find local (or global) optima.